

Jornada · 08 de mayo de 2026

Sesión #5 · ~3h de trabajo · agencia-web

Resumen ejecutivo

Sesión enfocada en blindar el dashboard del analizador frente a fallos de backend y construir un sistema de memoria del proyecto. Se desplegó la versión tolerante a fallos del dashboard, se confirmó que `/api/analyze` sigue dando 504 ocasional (Google PageSpeed lento), y se diseñó el sistema de bitácora + reportes diarios automáticos por email vía Edge Function de Vercel + cron + Resend. Se creó una skill 'agencia-analyzer' para consolidar el contexto del proyecto entre sesiones.

Trabajo realizado

✓ Dashboard tolerante a fallos de endpoints

Reescritura del flujo de fetch en `analyzer/index.html`: ahora usa `Promise.allSettled` con `AbortController` y timeout de 28s. Si `/api/analyze` falla, el dashboard usa los datos del crawl como fuente alternativa. Si `/api/crawl` falla, sigue con `analyze`. Si ambos fallan, error claro. Banner amarillo cuando hay datos parciales.

Archivos: `analyzer/index.html`

✓ Optimización de `/api/analyze` para evitar 504

Reescritura completa de `analyze.js`: timeout duro de 10s por cada llamada PageSpeed (mobile + desktop) usando `AbortController`. Las 6 tareas (PageSpeed mobile/desktop + HTML + robots + sitemap + llms.txt) lanzadas en paralelo. Garantía de respuesta total <22s.

Archivos: `api/analyze.js`

✓ Bug fix: scores 0/null en mini-cards

El HTML antiguo leía la estructura cruda de Lighthouse (`lighthouseResult.categories.X.score` 0-1) pero el filtro nuevo devuelve scores ya en 0-100 en `pagespeed.mobile.scores`. Bug corregido leyendo la nueva estructura. Adicionalmente, `hasAnalyze` más estricto para no mostrar 'null' cuando `analyze` responde sin scores.

Archivos: `analyzer/index.html`

✓ Zona DEBUG en el dashboard

Bloque colapsable al final del dashboard que muestra status, duration y JSON crudo de cada endpoint, con botones 'Copiar JSON' y 'Copiar todo'. Permite diagnosticar bugs en producción sin abrir DevTools ni buscar logs en Vercel.

Archivos: `analyzer/index.html`

✓ Skill 'agencia-analyzer' creada

Skill de Claude con `SKILL.md` (260 líneas) + 3 archivos de referencia (`lib-modules.md`, `dashboard-html.md`, `deployment.md`). Encapsula: estructura del repo, convenciones técnicas, shapes de

respuesta de los endpoints, las 8 dimensiones del Citability Score con pesos, paleta del dashboard, modelo de negocio, caso de prueba celes.ai, issues conocidos, workflow de deploy. Empaquetada como agencia-analyzer.skill (16KB) lista para instalar en Settings.

Archivos: `agencia-analyzer.skill`

✓ Diseño del sistema de bitácora

Definida arquitectura: BITACORA.md viva en docs/, ROADMAP.md con prioridades, DECISIONES.md con log cronológico, carpeta docs/reportes/ con PDFs diarios. Edge Function `/api/notify-report.js` con Vercel Cron (todos los días 19:00 UTC = 20:00 hora España) que envía email vía Resend con el PDF del día como adjunto a `albertodelgadohernando@gmail.com`.

Archivos: `docs/BITACORA.md` · `docs/ROADMAP.md` · `docs/DECISIONES.md` · `api/notify-report.js`

Decisiones tomadas

· Frontend tolerante a fallos en lugar de optimizar backend a toda costa

Contexto: `/api/analyze` daba 504 intermitente porque Google PageSpeed API tardaba >25s y Vercel Edge corta a los 25s.

Decisión: Hacer el dashboard tolerante a fallos (Promise.allSettled, banner de aviso, fallback a datos de crawl) Y optimizar analyze con timeouts de 10s.

Motivo: Doble defensa: aunque PageSpeed sea lento, el usuario nunca ve un error crudo, y el resto del análisis (más valioso para vender AEO/GEO) sigue funcionando.

· Sistema de bitácora con email diario automático

Contexto: El usuario quería evolución continua del proyecto sin perder contexto entre sesiones, con un resumen visible cada día.

Decisión: BITACORA.md viva en repo + reportes PDF diarios en docs/reportes/ + Edge Function con Vercel Cron que envía el PDF por email a las 20:00.

Motivo: Todo vive en el repo (versionado, compatible, no se pierde). Cron es nativo de Vercel (no necesita Make ni servicios externos). Resend gratis para 3.000 emails/mes.

· Skill 'agencia-analyzer' para consolidar contexto técnico

Contexto: Cada sesión nueva empezaba en blanco, perdiendo contexto y obligando a rehidratar info técnica.

Decisión: Crear skill instalable en Claude que carga automáticamente cuando se menciona el proyecto.

Motivo: Conocimiento estable (cómo está hecho) → skill. Estado cambiante (dónde estamos hoy) → bitácora. Combinación de ambos cubre todo.

Pendientes

Top prioridad próxima sesión

- Crear cuenta Resend y obtener API key (~3 min)
- Configurar variable de entorno `RESEND_API_KEY` en Vercel
- Subir todo el sistema de bitácora al repo y verificar primer envío de email

Backlog (sin urgencia)

- Probar el analizador con 2-3 sites más (stripe.com, una PYME ES, una Latam)
- Tier 2 backend: Mozilla Observatory + SSL Labs
- Tier 2 backend: DNS health (SPF/DMARC vía Cloudflare DoH)
- Tier 2 backend: Wayback Machine para frescura histórica
- Frontend persuasivo del informe (refinar copy, CTA, sección 'tu posición vs benchmark')
- Histórico en Supabase para Stage 3 (servicio recurrente)
- PDF descargable del informe del cliente
- Captar primer cliente piloto

Problemas encontrados

! Carpeta wrong al hacer git push

Resolución: El usuario estaba en C:\Users\sandr\Desktop\academy-e43af0.webflow (otro proyecto). Confirmamos que la correcta era C:\Users\sandr\Desktop\agencia-web.

! 504 GATEWAY_TIMEOUT en /api/analyze (PageSpeed lento)

Resolución: Mitigado en backend (timeouts 10s) y frontend (tolerancia + banner). No eliminado del todo: si PageSpeed va muy lento, el dashboard muestra solo crawl. Aceptable porque el valor diferencial está en AEO/GEO, no en velocidad.

! null en mini-scores cuando analyze respondía vacío

Resolución: hasAnalyze más estricto: solo true si pagespeed existe Y tiene scores numéricos válidos.

Aprendizajes

- Google PageSpeed API tarda con frecuencia >10s en sitios reales. No se puede confiar en respuesta rápida.
- Webflow exporta sites con 0 schemas JSON-LD por defecto. Mercado real para AEO/GEO.
- Skills de Claude son buenas para conocimiento estable; bitácoras en repo para estado cambiante. Necesitas ambas.
- Vercel Cron Jobs nativos eliminan la necesidad de Make/Zapier para tareas programadas.
- Promise.allSettled + AbortController es el patrón correcto para frontends que dependen de múltiples APIs externas.
- celes.ai como caso de prueba es excelente: PYME real, AI/SaaS, Latam, problemas técnicos típicos.

Conclusión

MVP del analizador desplegado y estable. Sistema de memoria del proyecto en producción. Lo siguiente es captación: probar con más sitios reales, refinar copy del informe persuasivo, y empezar conversaciones con primer cliente piloto. La infraestructura técnica deja de ser el cuello de botella; ahora toca el GTM.